

- Tomasulo算法实现
 - 原理
 - 代码说明及假设
 - Tomasulo部件之间异步执行模拟
 - 指令队列
 - 保留站
 - Load缓冲区和Store缓冲区
 - ROB
 - CDB
 - 结果展示
 - 推测执行改进Tomasulo算法的原理和实际效果的分析
 - 多发射超标量的优势和挑战
 - 总结

Tomasulo算法实现

学号：22320021 姓名：陈安康

原理

Tomasulo算法是一种动态调度技术，它通过“按序发射，乱序执行”的方式提高处理器性能，并利用重排序缓冲区（ROB）确保结果“顺序提交”。该算法主要依靠寄存器重命名技术来解决数据冲突，允许指令在没有直接数据依赖的情况下并行执行。

代码说明及假设

代码组织我采用模块化组织形式，每一个模块编写为一个类。包括保留站（RS）、Load缓冲区（Load_buffer）、Store缓冲区（Save_Buffer）、CDB、ROB、指令队列（Instruction_Queue）。Result文件中存放一些全局变量，包括结果表，推测执行开关、访存锁；Clock中存放时钟；Tomasulo.py文件为运行主文件。

代码基于假设如下：

1) 保留站和缓冲区从ROB和CDB两者中获取数据。2) 当保留站、缓冲区、ROB、指令队列满时，系统阻塞，直到有空余空间。3) 假设浮点除法和乘法共用一个功能单元。4) 假设发射阶段需要一个周期，写回阶段需要一个周期。5) 由于要求中没有对整数计算以及访存操作的时钟周期数进行要求，这里假设为整数计算为2个时钟周期，访存操作花费2个时钟周期。其余按照要求。6) 指令队列每周期读入的指令数目和发射的指令数目一致。这样即可保证不会使读入指令成为影响性能的因素。7) 访问内存的端口只有一个，每一时刻最多有一个Load和Store指令执行。8) 浮点数访存和整数访存一致

8) 假设访问内存永不会冲突，其合理性解释如下：

对于第一段两个Load指令，由于计算地址所需寄存器不同，这里假定访问的地址也不同是合理的。

对于第二段代码，其访问地址在同一个迭代周期内相同，关键点在于在同一个迭代周期下会不会出现Load和Store同时执行的情况，实际上这种情况是不可能发生的，因为Store需要等待addi的计算结果，而addi的计算执行需要Load的执行结果，所以导致Load和Store在同一个迭代周期下访存操作是有序的，这样，在同一个迭代周期内不会冲突，而在不同迭代周期中访存地址不同，所以不会冲突。

因此，代码中不会有对内存的一致性的判断，一律认为内存不会冲突。

Tomasulo部件之间异步执行模拟

为了模拟各个部件的异步执行，这里采用的方式是各部件先进行一次更新（代码中的监听主要是为了在ROB场景下，某些保留站或缓冲区从ROB获取值），如果此时有部件写CDB，那么所有部件会重新监听CDB，再次尝试更新。代码设计保证每个部件在两次更新中最多进行一次更新，保证正确性。通过这种方式，就可以模拟在一个周期内的异步运行情况。

指令队列

指令队列主要作用是读取指令.txt文件，对原始指令字符串进行处理，实现发射阶段的相关操作。对于多发射，指令队列中通过issue_count来指示一次发射多少条指令，通过改变这个数值，以及与之配套的顺序检查、恢复机制的实现来保证超标量发射的实现正确。

对于恢复机制，由于一条指令要填入ROB、保留站、缓冲区，如果其中任意一个由于部件满了填入失败（在此代码中只有可能是这个原因），那么其余已经成功填入的部件都要回滚，这里在ROB和缓冲区中的fill函数里面实现了一个反馈，若填入成功返回序号，否则返回-1，通过这个序号，若某一项填入失败，则可以调用某一部件相应的函数对其填入内容进行删除。实现回滚。部分代码示例如下：

```
index2 = rob.fill(words)
    # 判定顺序不能变！！依靠代码短路来保证正确性！！
    if index2 != -1 and
reservationstations.fill(words,registers,timer,index2,rob):
    # 成功填入
    Result.ans[Result.index] =
[self.instruction\_queue\[0\],clock,-1,-1,-1]
    Result.index += 1
    del self.instruction\_queue\[0\]
else:
    rob.clear(index2)
    return
```

顺序检查保证代码发射是顺序发射的，而且ROB是依据发射顺序作为其提交顺序的，因此顺序检查是十分重要的。这里的指令队列每次都从头部读取指令，发射成功就弹出，当出现发射失败的情况时，立即终止此周期的剩余发射任务，从而保证所有指令都是顺序发射。

推测执行是顺其自然的，虽然它在实际硬件中是依靠ROB来实现的，但在代码模拟中，这个功能和ROB是解耦的。代码中ROB的唯一主要的作用就是保证顺序提交。如果不对BNE做特殊的处理，那么bne只是会被当做平常指令经过，从而实现推测执行的效果。想要不实现推测执行的效果，反而需要把bne指令进行特殊的判断和处理。为了实现这个效果，我在Result中定义了一个发射锁，当

检测到当前处理指令是bne指令时，若开启推测执行，那么就用锁阻塞指令队列发射，直到bne指令执行完成代码部分如下：

Result.py中的定义：

```
SE = True # 这个是否启动推测执行的
SE_H = True # 这个是实际操作推测执行的变量
```

指令队列中的代码：

```
f op == "bne":
    # 不启用推测执行
    if Result.SE == False:
        # 关闭issue功能，不再发射指令，直到执行完成
        Result.SE_H = False
```

当保留站中执行完成后：

```
else:
    # 解除发射限制
    Result.SE_H = True
```

保留站

保留站仿照真实的保留站进行组织，每一个保留站项包括timer、status、op、qk、qj、vk、vj、result、op_clock等结构。支持填入、监听、更新、清空等操作。更上层将保留站项组织成浮点数加法（fa）、浮点数乘法（fm）、整数加法（iu）、分支预测（bne）、地址计算（sd_ld）保留站，供fpu使用。为了方便，fpu的状态以及使用者信息也一并包含在里面。

下面介绍基本的操作代码：

对于填入操作，会依据指令队列发射的指令按要求填入各个结构，并且访问寄存器，获取当前数据是否处于可以获取的状态。主要的代码如下：

```
def fill(self, words, registers, timer, result, rob):
    self.status = 'wait'
    self.timer = timer
    # 获取操作字
    self.op = words[0]
    # 判断是不是寄存器,这里分别对应常数, 寄存器已经是最新值, 寄存器不是最新值, 但是ROB
    # 中结果已经计算, 只是没有提交
    if words[2] not in registers.registers or registers.registers[words[2]] == '':
        or (rob.items[registers.registers[words[2]]].status == 'ready'):
            self.vj = words[2]
        else:
            self.qj = registers.registers[words[2]]
            if words[3] not in registers.registers or registers.registers[words[3]] == '':
```

```

    ' or (rob.items[registers.registers[words[3]]].status == 'ready'):
        self.vk = words[3]
    else:
        self.qk = registers.registers[words[3]]
    # 更新保留站的结果
    self.index = Result.index
    self.result = result
    # 对寄存器组状态的更新
    # load由缓冲区自己更新，store不更新
    if not(self.op == "ld" or self.op == "fld" or self.op == "sd" or self.op ==
"fsd"):
        if words[1] in registers.registers:
            registers.registers[words[1]] = result

```

更新操作是另一项主要操作，其主要思路是**通过状态机依据status指示的状态在每个时钟周期对保留站项进行相应的操作**，最主要的状态是从等待变为运行以及从运行变成结束时的一系列操作，代码如下：

```

def update(self, fcu_status, cdb, rob = False):
    if self.op_clock != Clock.clock:
        if self.status == 'wait':
            if self.qj == '' and self.qk == '' and fcu_status == 'idle':
                # 这里到底要不要减1还是要考虑考虑
                Result.ans[self.index][2] = Clock.clock + 1
                # 这里的减1在硬件上没有逻辑，纯粹是为了使bne store指令这些不写CDB的
                # 指令在观感上跳过写CDB的停顿
                if self.op == 'bne':
                    self.timer -= 1
                self.status = 'run'
                # self.timer -= 1
                fcu_status = 'run'
                self.op_clock = Clock.clock
                return 1, fcu_status# 修改了fcu
            return -1, fcu_status
        # 为了逻辑正确，这个要放减法前面
        # 执行结束
        if self.status == 'run' and self.timer == 0:
            # cdb 获取数据
            # bne没有写回阶段
            if self.op != 'bne':
                if self.op != 'sd' or self.op != 'fsd':
                    Result.ans[self.index][4] = Clock.clock
                if cdb.write(self.result):
                    # 写表格
                    # 清空
                    fcu_status = 'idle'
                    self.clear()
                    return 2, fcu_status
            else:
                # 解除发射限制
                Result.SE_H = True
                rob.items[self.result].status = 'ready'
                fcu_status = 'idle'
                self.clear()
                return 2, fcu_status

```

```

        return -1, fcu_status # 告知当前fcu没人使用，把使用者设为-1
    if self.status == 'run':
        if self.timer != 0:
            self.timer -= 1
            self.op_clock = Clock.clock
            return -1, fcu_status
        # 已经执行完成
        return -1, fcu_status
    else:
        return -1, fcu_status

```

监听操作用以监听CDB和ROB（如果无ROB的场景下，ROB监听部分实际不会起作用，原理ROB的部分会介绍），更新qk和qj：

```

# 监听CDB
def listen(self, cdb, rob):
    # 插cdb
    if self.qj == cdb.broadcast():
        self.vj = cdb.broadcast()
        self.qj = ''

    if self.qk == cdb.broadcast():
        self.vk = cdb.broadcast()
        self.qk = ''

    # 查rob
    if self.qj != '' and rob.items[self.qj].status == 'ready':
        self.vj = rob.items[self.qj].result
        self.qj = ''

    if self.qk != '' and rob.items[self.qk].status == 'ready':
        self.vk = rob.items[self.qk].result
        self.qk = ''

```

另外的说明：

在保留栈或者缓冲区中有result结构，其保存结果要写入的指令在ROB的序号（即使没有ROB的场景下，此序号依旧可以起到辨识作用，后面ROB的部分会解释），当执行完成，会把result写入CDB进行广播，以此来通知保留站、缓冲区、ROB。

由于bne指令不写CDB，所以这两种指令直接改ROB来更新ROB中指令的状态。并且由于它们没有写CDB阶段，但是由于保留站项和缓冲区是统一代码编写的，所以为了平衡掉预留的写CDB阶段，这里对Store指令（在缓冲区）和bne指令在实际执行的前一个周期就对计时器减1。

update中op_clock的作用是防止同一个周期多次执行update操作。保证运行的正确性。

Load缓冲区和Store缓冲区

基于“访问内存的端口只有一个，每一时刻最多有一个Load和Store指令执行。”的假设，所以在Result.py文件中定义访存锁，用来保证每个时刻只能有一个操作来执行访存操作。

Load缓冲区数据结构由status、address、timer、result、op_clock等组成。在指令队列发射指令后，该指令会尝试填入ROB、Load (Store) 缓冲区、和地址计算保留站里面，Load缓冲区在CDB上（这里我设计的地址计算保留站的结果不会最后写入ROB，仅在CDB广播供Load和Store指令获取）获取计算得到的地址，然后尝试获取访存锁，若成功则执行访存操作，Load指令执行完后将结果写CDB，Store指令由于不写CDB，所以直接修改ROB。

Load缓冲区和Store缓冲区的项数据结构由status、address、timer、result、op_clock等组成。同样支持填入、监听、更新、清空等操作。每个操作的实现逻辑和保留站一致，由于报告页数限制，这里就不重复赘述。

ROB

ROB实现了转发功能。保留站和缓冲区可以及时从ROB读取还没有提交到寄存器的数据。每一个项实现填入、提交、监听、清除功能（实现思路也与保留站一致，限于篇幅不展开了），并在上层组织成一个回环队列，通过维护一个头部指针进行统一管理，在提交时对头部进行判断，只有在头部ready时提交头部，依次往下，实现顺序提交的效果。由于指令预测永远正确，所以这里没有设计分支预测错误时的多指令回滚机制。

```
def commit(self, registers):
    while self.items[self.start].status == 'ready':
        self.items[self.start].commit(registers)
        self.start = (self.start + 1) % self.size
```

在非推测执行中，没有ROB模块，但是在写代码时已经将ROB代码高度耦合在其他模块中。这里对于非推测执行，我对代码进行较少量修改来消除ROB的影响：1) 将ROB容量调至无穷大，避免其容量对发射产生影响，这里的指令区分然后把ROBcommit函数修改寄存器的部分注释掉，这样取消对寄存器的影响，采用CDB广播来更新寄存器。ROB的转发操作主要是解决寄存器更新不及时导致保留站无法及时获取最新数值的问题，而当寄存器采用CDB及时更新时，那么这部分问题就不存在了，因此保留站本身就可以依靠CDB即可获取最新数据，**ROB在此时还在运行但对其他部件和结果不起任何作用了**。起到了消除ROB的作用。

CDB

CDB的实现比较简单，主要是接受各个部件的结果并且进行广播。在写CDB的过程中可能会发生冲突，即多个部件在同一个周期同时写CDB的问题，这里我在写入时进行了反馈，若CDB在此周期已经被写入数据，则返回False，告知相关部件写入失败，则相关部件接受到反馈后依据反馈进行相关处理。保留站占据FCU等下一个周期尝试写入，Load和Store占据锁，等下一个周期尝试写入。

```
def write(self, data):
    if self.data == '':
        self.data = data
        return True
    else:
        return False
```

代码中的值说明：

在寄存器中，空值（"）表示该寄存器是最新的值，可以直接取用，若为数字a，表示该寄存器正等待在ROB中的序号为a的指令的结果。在保留站中，主要是对qk和qj的判断，当qk或qj是空值（"）时，表示数据已经准备完毕，当qk和qj是数字a，则表示其在等待ROB中的序号为a的指令的结果。Load和Store缓冲区中初始填入的地址为“数字_sd（ld）”的形式，数字表示该指令在ROB中的序号，ld或者sd表示操作。表明在等待整数功能单元地址计算结果，对应的整数功能单元在完成计算后通过CDB广播告知其计算完成，此时地址修改为空值（"），表示已经获取。

对于代码中结果的执行阶段和写CDB阶段的说明：

在代码中，当保留站（缓冲区）中的指令在某一个周期所有数据准备完成，且FCU处于空闲状态，此时指令会占据FCU，并在下一个时钟周期执行，实际执行开始的周期被记录为执行开始周期。比如一条在缓冲区的Load指令访问内存需要2个时钟周期，它在第3周期地址和数据准备完毕，此时计时器设置为2，**将第4周期（实际执行开始周期）作为指令执行周期进行记录**，然后经历第4周期，计时器变为1，第5周期，计时器变为0，执行完毕，**第6周期写CDB**。

结果展示

所有结果记录开始周期。-1表示此指令没有该阶段，Load和Store指令都会进行地址计算，**地址计算的执行开始周期和写CDB周期都会写到结果中**，所以它们的执行阶段的周期即地址计算开始执行的周期数，而Store指令的写CDB的周期即地址计算写CDB的周期。

Result.ans里面的每一个表项的组成是键值表示第几个指令，列表中依次是指令字符串、发射周期、执行开始周期、访存开始周期、写CDB周期。

这里对存储指令的.txt文件中指令的格式有要求，**寄存器之间不能用逗号隔开，要用空格。**

单发射不支持推测：

```
PS C:\Users\86135\Desktop\计算机体系结构\大作业> & "C:/Program Files/Python312/python.exe" c:/Users/86135/Desktop/计算机体系结构/大作业/Tomasulo.py
{0: ['fld f6 32(x2)\n', 0, 1, 4, 6], 1: ['fld f2 44(x3)\n', 1, 4, 8, 10], 2: ['fmul.d f0 f2 f4\n', 2, 11, -1, 17], 3: ['fsub.d f8 f2 f6\n', 3, 11, -1, 13], 4: ['fdiv.d f0 f0 f6\n', 4, 18, -1, 30], 5: ['fadd.d f6 f8 f2', 5, 14, -1, 16]}
```

将上面的结果整理成表格如下：

迭代数	指令	发射	执行	访存	写CDB
1	fld f6 32(x2)	0	1	4	6
1	fld f2 44(x3)	1	4	8	10
1	fmul.d f0 f2 f4	2	11		17
1	fsub.d f8 f2 f6	3	11		13
1	fdiv.d f0 f0 f6	4	18		30
1	fadd.d f6 f8 f2	5	14		16

单发射支持推测：

```
PS C:\Users\86135\Desktop\计算机体系结构\大作业> & "C:/Program Files/Python312/python.exe" c:/Users/86135/Desktop/计算机体系结构/大作业/Tomasulo.py
{0: ['fld f6 32(x2)\n', 0, 1, 4, 6], 1: ['fld f2 44(x3)\n', 1, 4, 8, 10], 2: ['fmul.d f0 f2 f4\n', 2, 11, -1, 17], 3: ['fsub.d f8 f2 f6\n', 3, 11, -1, 13], 4: ['fdiv.d f0 f0 f6\n', 4, 18, -1, 30], 5: ['fadd.d f6 f8 f2', 5, 14, -1, 16]}
PS C:\Users\86135\Desktop\计算机体系结构\大作业>
```

整理成表格如下：

迭代数	指令	发射	执行	访存	写CDB
1	fld f6 32(x2)	0	1	4	6
1	fld f2 44(x3)	1	4	8	10
1	fmul.d f0 f2 f4	2	11		17
1	fsub.d f8 f2 f6	3	11		13
1	fdiv.d f0 f0 f6	4	18		30
1	fadd.d f6 f8 f2	5	14		16

双发射不支持推测（执行29个周期，发射了20条指令）：

```
PS C:\Users\86135\Desktop\计算机体系结构\大作业> & "C:/Program Files/Python312/python.exe" c:/Users/86135/Desktop/计算机体系结构/大作业/Tomasulo.py
{0: ['Loop: ld x2 0(x1)\n', 0, 1, 4, 6], 1: ['addi x2 x2 1\n', 0, 7, -1, 9], 2: ['sd x2 0(x1)\n', 1, 4, 10, 7], 3: ['addi x1 x1 8\n', 1, 2, -1, 4], 4: ['bne x1 x3
Loop', 2, 5, -1, -1], 5: ['Loop: ld x2 0(x1)\n', 7, 8, 12, 14], 6: ['addi x2 x2 1\n', 7, 15, -1, 17], 7: ['sd x2 0(x1)\n', 8, 11, 18, 13], 8: ['addi x1 x1 8\n', 8,
10, -1, 12], 9: ['bne x1 x3 Loop', 9, 13, -1, -1], 10: ['Loop: ld x2 0(x1)\n', 15, 16, 20, 22], 11: ['addi x2 x2 1\n', 15, 23, -1, 25], 12: ['sd x2 0(x1)\n', 16,
19, 26, 21], 13: ['addi x1 x1 8\n', 16, 18, -1, 20], 14: ['bne x1 x3 Loop', 17, 21, -1, -1], 15: ['Loop: ld x2 0(x1)\n', 23, 24, 28, 26], 16: ['addi x2 x2 1\n', 23,
-1, -1, -1], 17: ['sd x2 0(x1)\n', 24, 27, -1, 29], 18: ['addi x1 x1 8\n', 24, 26, -1, 28], 19: ['bne x1 x3 Loop', 25, 29, -1, -1]}
PS C:\Users\86135\Desktop\计算机体系结构\大作业>
```

（序号15的Load指令中写CDB是计算地址完成时写CDB，若访存写CDB完成会进行覆盖，这里是
因为29个周期太短使Load指令没执行完，所以还没来得及覆盖，不是错误，特此说明）由于太
多，这里统一列出两轮迭代的表格。

再次重申！Store指令的写CDB的周期即地址计算写CDB的周期

迭代数	指令	发射	执行	访存	写CDB
1	Loop: ld x2 0(x1)	0	1	4	6
1	addi x2 x2 1	0	7		9
1	sd x2 0(x1)	1	4	10	7
1	addi x1 x1 8	1	2		4
1	bne x1 x3 Loop	2	5		
2	Loop: ld x2 0(x1)	7	8	12	14
2	addi x2 x2 1	7	15		17
2	sd x2 0(x1)	8	11	18	13
2	addi x1 x1 8	8	10		12
2	bne x1 x3 Loop	9	13		

由于第一轮迭代bne在第5周期开始执行，在第6周期执行完成（执行了5，6两个周期），所以第7
周期第二轮指令才能够发射。

双发射支持推测（执行了29个周期，发射了26条指令）：

```
PS C:\Users\86135\Desktop\计算机体系结构\大作业> & "C:/Program Files/Python312/python.exe" c:/Users/86135/Desktop/计算机体系结构/大作业/Tomasulo.py
{0: ['Loop: ld x2 0(x1)\n', 0, 1, 4, 6], 1: ['addi x2 x2 1\n', 0, 8, -1, 10], 2: ['sd x2 0(x1)\n', 1, 4, 11, 8], 3: ['addi x1 x1 8\n', 1, 2, -1, 4], 4: ['bne x1 x3
Loop', 2, 5, -1, -1], 5: ['Loop: ld x2 0(x1)\n', 2, 9, 13, 15], 6: ['addi x2 x2 1\n', 3, 17, -1, 19], 7: ['sd x2 0(x1)\n', 3, 12, 21, 14], 8: ['addi x1 x1 8\n', 4,
5, -1, 7], 9: ['bne x1 x3 Loop', 4, 8, -1, -1], 10: ['Loop: ld x2 0(x1)\n', 7, 15, 18, 20], 11: ['addi x2 x2 1\n', 11, 21, -1, 23], 12: ['sd x2 0(x1)\n', 13, 18,
24, 21], 13: ['addi x1 x1 8\n', 13, 14, -1, 16], 14: ['bne x1 x3 Loop', 14, 17, -1, -1], 15: ['Loop: ld x2 0(x1)\n', 16, 22, 26, 28], 16: ['addi x2 x2 1\n', 20, 3
0, -1, -1], 17: ['sd x2 0(x1)\n', 23, 25, -1, 27], 18: ['addi x1 x1 8\n', 23, 24, -1, 26], 19: ['bne x1 x3 Loop', 24, 27, -1, -1], 20: ['Loop: ld x2 0(x1)\n', 24,
28, -1, -1], 21: ['addi x2 x2 1\n', 25, -1, -1, -1], 22: ['sd x2 0(x1)\n', 26, -1, -1, -1], 23: ['addi x1 x1 8\n', 26, 27, -1, 29], 24: ['bne x1 x3 Loop', 27, 30,
-1, -1], 25: ['Loop: ld x2 0(x1)\n', 29, -1, -1, -1]}
PS C:\Users\86135\Desktop\计算机体系结构\大作业>
```

整理表格如下：

迭代数	指令	发射	执行	访存	写CDB
1	Loop: ld x2 0(x1)	0	1	4	6
1	addi x2 x2 1	0	8		10
1	sd x2 0(x1)	1	4	11	8
1	addi x1 x1 8	1	2		4
1	bne x1 x3 Loop	2	5		
2	Loop: ld x2 0(x1)	2	9	13	15
2	addi x2 x2 1	3	17		19
2	sd x2 0(x1)	3	12	21	14
2	addi x1 x1 8	4	5		7
2	bne x1 x3 Loop	4	8		

由于ROB大小设为了10，所以在存储完10条指令后，指令队列就会由于ROB已满阻塞。直到前面的指令有提交才会继续发射。

由于只列出了结果中前两轮迭代的数据，他们有些指令受到了后续指令的影响，如第8条Store指令的执行由于后续指令占用访存端口而延迟，第7条addi指令由于后续指令占用FCU而延后执行等。如果想要分析透彻要结合图片中列出的全部的执行情况来分析。

推测执行改进Tomasulo算法的原理和实际效果的分析

对于第一个任务，由于指令中不涉及分支指令，导致在结果表格中二者没有区别。不过由于ROB的存在，使得**提交指令变得有序**（二者这方面的变化可以比较output1.txt和output_rob.txt），相比于无推测执行时的乱序提交，这可以**维持系统精确的中断和异常**，提高系统执行错误时恢复的能力。

对于第二个任务，相比于无推测执行，推测执行在相同时间下发射的指令数更多，经计算，在29个周期中执行完成19条指令，IPC为0.655。而无推测执行场景下，29个周期完整执行了16条指令，IPC为0.551。相比于无推测执行有小部分提升，但没有预期的那么高。这是因为1) 虽然**推测执行使得系统不用等待分支指令执行结束就执行接下来的指令，缩减了这一部分的周期延迟**，但却给部件带来了更多的竞争，许多指令由于部件被其他指令占用而被迫阻塞，导致实际提速没有预期那么多。2) ROB的规模很大程度上阻碍了效率的提高，尤其在双发射场景下，ROB填满迫使指令发射阻塞，也一定程度上拖慢了效率。

综上所述，ROB使指令按序提交，给系统维持精确中断和异常提供了条件；使得分支指令不必得到结果就可以通过推测继续执行，若预测错误则在ROB中可以按序回滚，为推测执行提供硬件支持。在效率上也有提升，但ROB的大小和部件的竞争会成为效率提升的桎梏因素。

多发射超标量的优势和挑战

计算四组实验的IPC如下：

单发射无推测执行和单发射有推测执行执行效果一致，在31个周期（因为是从0开始计数的）内完整执行完6条指令， $IPC = 6 / 31 = 0.194$

双发射无推测执行，在29个周期完整执行16条指令， $IPC = 16 / 29 = 0.551$

双发射有推测执行，在29个周期完整执行19条指令， $IPC = 19 / 29 = 0.655$

从结果上看，双发射将效率提高了两倍甚至三倍，但其实这么比较没有意义，因为**两个任务的指令片段不一样**，一个执行周期更大的浮点数运行，另一个以访存和整数计算为主。因此，为了比较客观，将第一个任务在双发射场景下运行，结果如下：

```
PS C:\Users\86135\Desktop\计算机体系结构\大作业> & "C:/Program Files/Python312/python.exe" c:/Users/86135/Desktop/计算机体系结构/大作业/Tomasulo.py
{0: ['fld f6 32(x2)\n', 0, 1, 4, 6], 1: ['fld f2 44(x3)\n', 0, 4, 8, 10], 2: ['fmul.d f0 f2 f4\n', 1, 11, -1, 17], 3: ['fsub.d f8 f2 f6\n', 1, 11, -1, 13], 4: ['fd
iv.d f0 f0 f6\n', 2, 18, -1, 30], 5: ['fadd.d f6 f8 f2', 2, 14, -1, 16]}
PS C:\Users\86135\Desktop\计算机体系结构\大作业>
```

可以看到，对比任务一，除了发射提前之外，没有任何改进，为什么会这样呢？分析原因就会发现，双发射带来更多的部件竞争，而且任务1的指令中存在很多写后读的真数据相关，导致很多指令即使过早发射，也会由于部件被其他指令占用（Load指令地址计算FCU的抢占，以及写CDB时CDB冲突导致第二个ld指令的地址写CDB延后一个时钟周期等），数据等待（浮点乘法和浮点减法指令等待Load指令执行结果等）导致性能没有多少提升。

在任务二下，运行单发射有推测执行，结果如下：

```
PS C:\Users\86135\Desktop\计算机体系结构\大作业> & "C:/Program Files/Python312/python.exe" c:/Users/86135/Desktop/计算机体系结构/大作业/Tomasulo.py
{0: ['Loop: ld x2 0(x1)\n', 0, 1, 4, 6], 1: ['addi x2 x2 1\n', 1, 8, -1, 10], 2: ['sd x2 0(x1)\n', 2, 4, 11, 8], 3: ['addi x1 x1 8\n', 3, 4, -1, 7], 4: ['bne x1 x3
Loop', 4, 8, -1, -1], 5: ['Loop: ld x2 0(x1)\n', 5, 12, 15, 17], 6: ['addi x2 x2 1\n', 6, 19, -1, 21], 7: ['sd x2 0(x1)\n', 7, 9, 23, 11], 8: ['addi x1 x1 8\n', 8
, 11, -1, 13], 9: ['bne x1 x3 Loop', 9, 14, -1, -1], 10: ['Loop: ld x2 0(x1)\n', 10, 15, 20, 22], 11: ['addi x2 x2 1\n', 11, 23, -1, 25], 12: ['sd x2 0(x1)\n', 13,
20, 26, 23], 13: ['addi x1 x1 8\n', 14, 15, -1, 18], 14: ['bne x1 x3 Loop', 15, 19, -1, -1], 15: ['Loop: ld x2 0(x1)\n', 18, 24, 28, 26], 16: ['addi x2 x2 1\n', 2
2, -1, -1, -1], 17: ['sd x2 0(x1)\n', 25, 27, -1, 29], 18: ['addi x1 x1 8\n', 26, 27, -1, 29], 19: ['bne x1 x3 Loop', 27, 30, -1, -1], 20: ['Loop: ld x2 0(x1)\n',
28, -1, -1, -1], 21: ['addi x2 x2 1\n', 29, -1, -1, -1], 22: ['sd x2 0(x1)\n', 30, -1, -1, -1]}
```

29个时钟周期完整执行了17个指令， $IPC = 17 / 29 = 0.586$ 。

对比任务二下双发射有推测执行，可以看见双发射实际效率提升并没有这么多，从上面的实验过程我们可以看到，**多发射会增加部件冲突的概率，对硬件的处理能力（并行性、容量等）提出了更多要求**。并且**多发射提升效率要求指令之间没有那么多的真数据相关**，否则即使过早发射指令，指令还是因为数据不能早到位而被迫阻塞。

通过以上分析可以看到，多发射超标量确实可以在某些情况下提升效率，但与此同时其对部件的要求也会相应提升，同时在某些特定指令片段下，如大量真数据相关，多发射并不能提高效率。

总结

通过这次实验，我成功构建了Tomasulo算法的模拟器，并根据实验结果深入分析了Tomasulo算法中的推测执行和超标量技术。这一过程不仅加深了我对Tomasulo算法原理和实现细节的理解，还让我对推测执行和超标量技术的实际应用有了更深刻的认识，收获颇丰